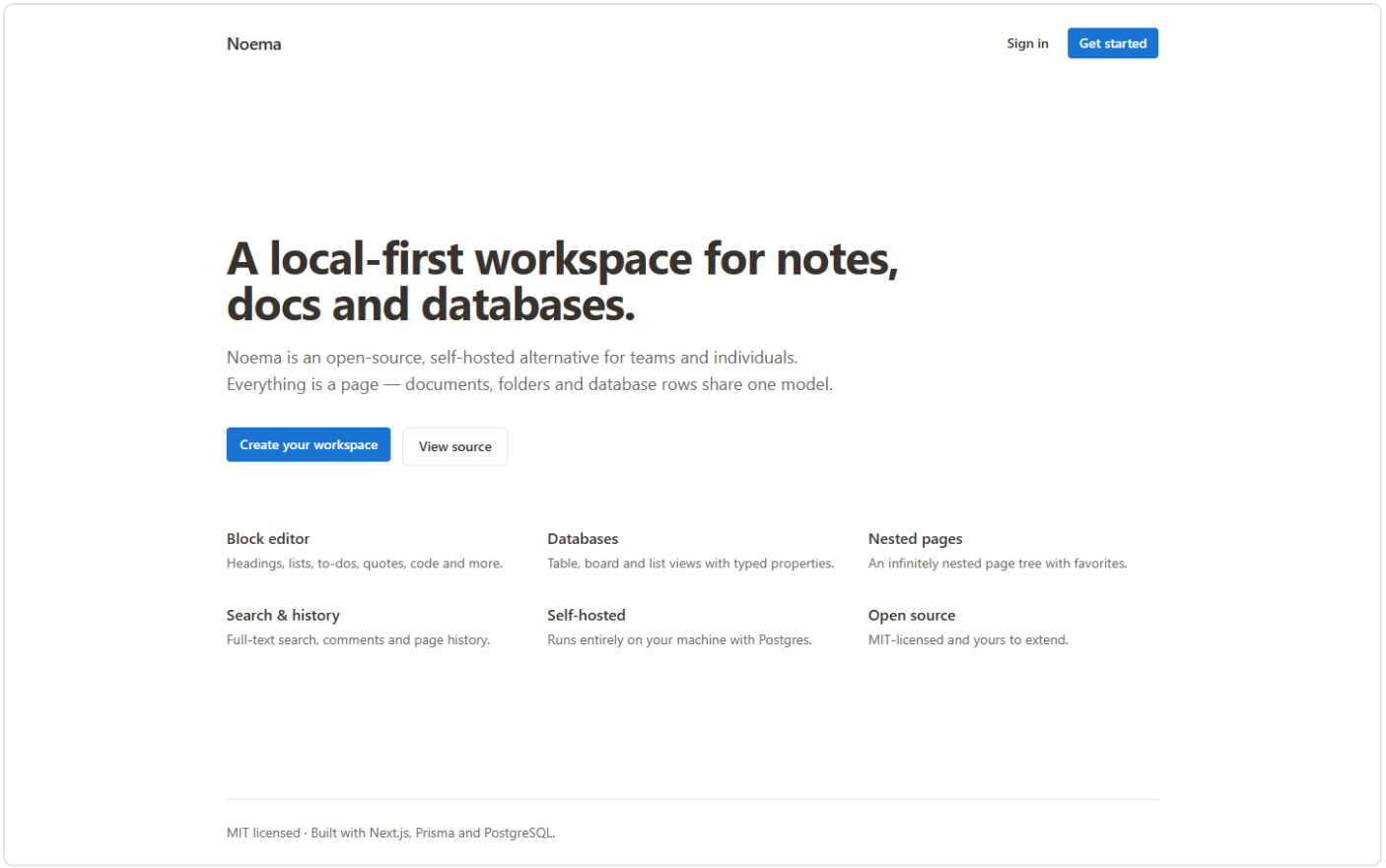
◆ **Noema**

Feature overview Version 1.4.1 24 October 2026 MIT licence

A local-first workspace for notes, docs and databases.

Noema is an open-source workspace — the editor, the databases, the tracker and the desktop app — built end to end. This document is the tour: what it does, grouped by area, illustrated with screenshots taken from the seeded demo workspace.



The public landing page — /

Authors
Bilel Guembri and Wissal Ben Othmen

Source
github.com/Guembri01/noema

Screenshots
44, generated by the repo

At a glance

Every feature area is tracked row by row in the repository, with a status and a note explaining the current state. Nothing is dropped silently.

563	454	87	44
tracked feature rows	implemented	partial, enumerated	screenshots in this document

0 planned · 22 deliberately out of scope. The gap list is generated from the same source as the tracker, so the two cannot disagree.

What’s inside

01 Getting started

02 Writing — the block editor

03 Databases and views

04 Planning — tickets, cycles and projects

05 Finding and organising

06 Collaboration

07 Sharing and permissions

08 Platform, AI and administration

09 The desktop app

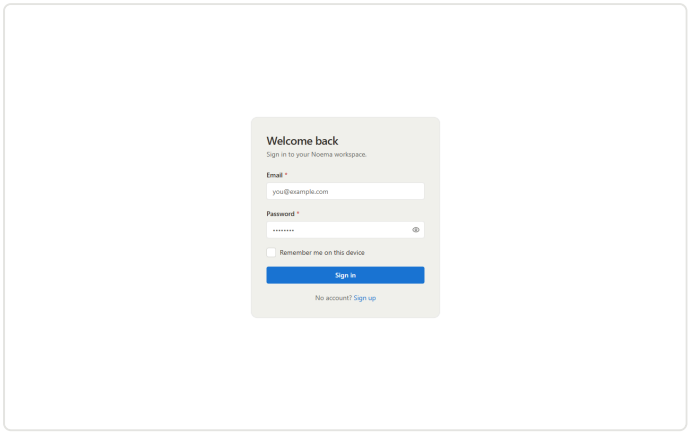
10 Quality and engineering

CHAPTER 01

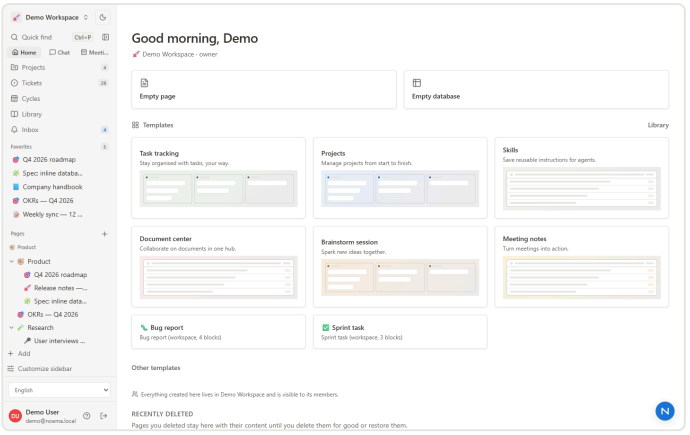
Getting started

Sign in, land in a workspace, and move around it without reaching for the mouse.

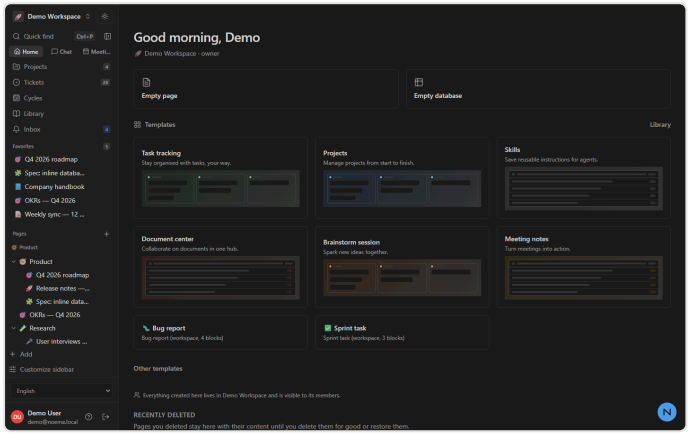
- **Auth** — Email + password signup/login/logout, hashed passwords (bcrypt), signed JWT session cookie
- **Workspaces** — Create workspaces, memberships + roles (OWNER/ADMIN/MEMBER/GUEST), multi-workspace switcher
- **Pages** — Infinitely nested page tree, icons, favorites, soft-delete/trash, version snapshots
- **Navigation** — **Row side-peek**, **responsive mobile shell** (drawer sidebar), ⌘K quick find, breadcrumbs, following/favorites
- **Sidebar chrome** — **Home / Chat / Meetings** tabs, **Projects**, **Library**, **Teamspaces**, **Private pages**, search, **Add**, **Customize**, a footer help menu, and a **right-click menu on every page** (open, open in new tab, discuss in chat, rename, favorite, copy link, move to teamspace/project, trash)
- **UX** — **Notion-white default theme** (light) with a dark toggle, **⌘K quick-find command palette**, toast notifications, tooltips, responsive sidebar, accessible primitives (Radix)



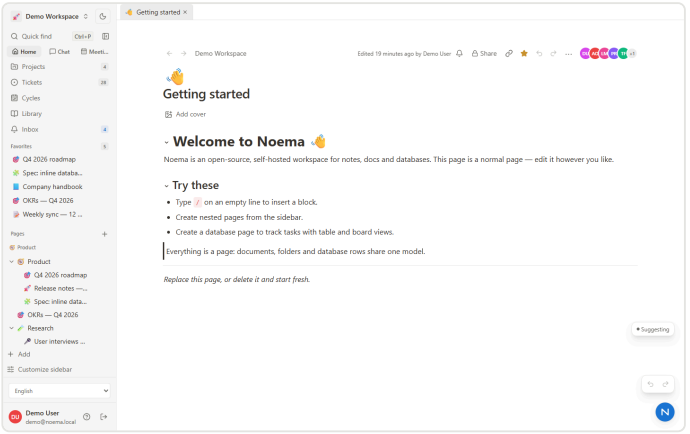
Sign in — email, password, remember-me
`/login`



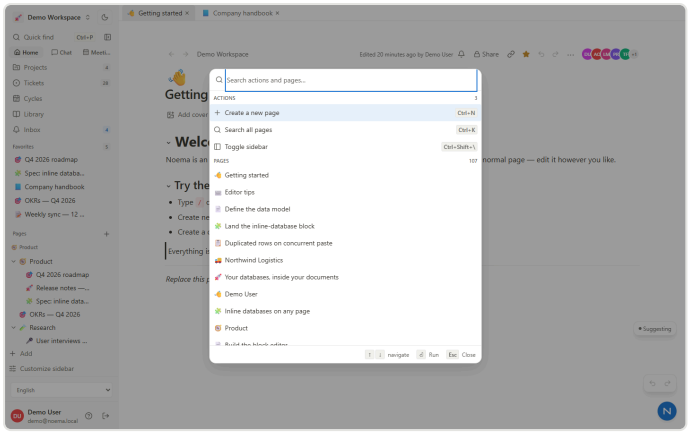
Workspace home — quick starts and the template gallery
`/w/demo-workspace`



The same home in dark mode
`/w/demo-workspace`

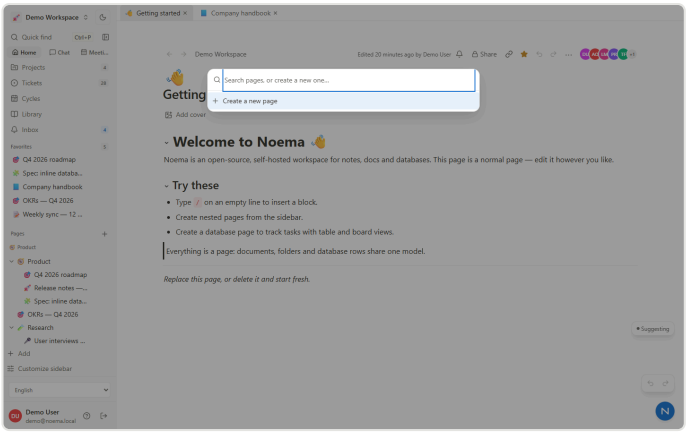


Sidebar — favourites, pages, teamspaces, projects, tickets
`/w/demo-workspace/cmvbj2631002hukloi0z5s75o`



Command palette (Ctrl/⌘ K)

/w/demo-workspace/cmvbj2631002hukloi0z5s75o



Quick find (Ctrl/⌘ P)

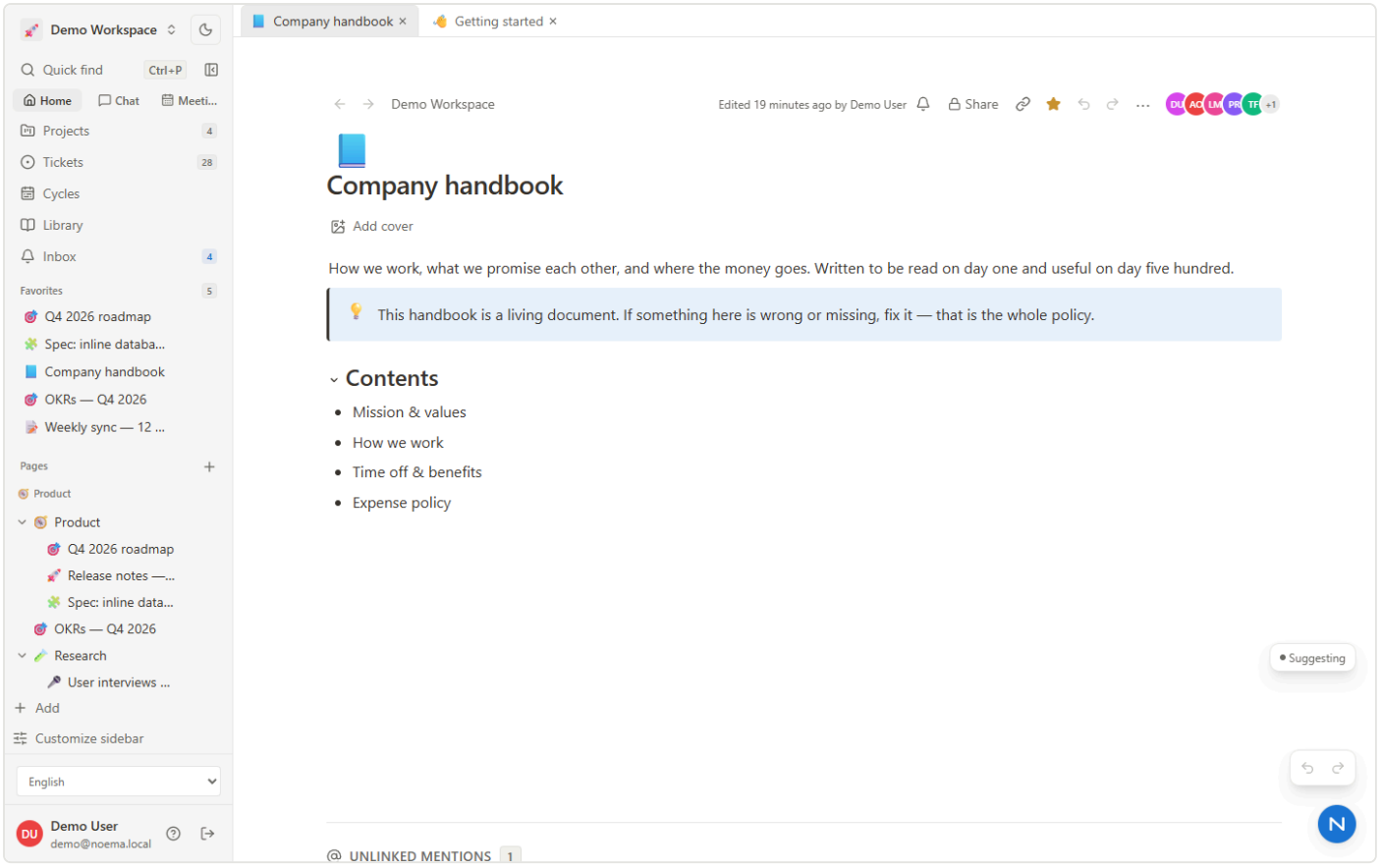
/w/demo-workspace/cmvbj2631002hukloi0z5s75o

CHAPTER 02

Writing — the block editor

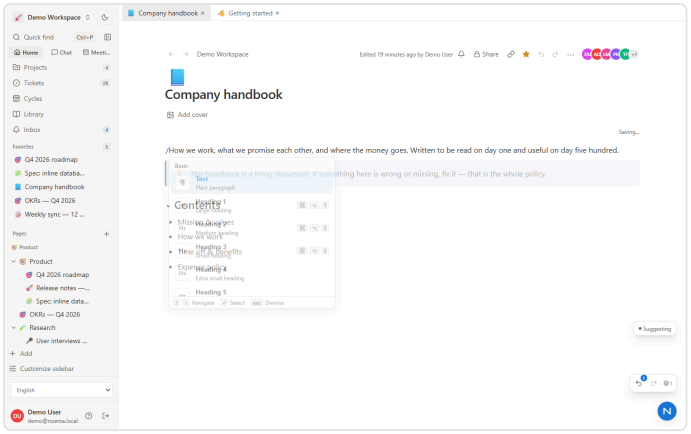
A TipTap editor with Notion's block vocabulary, built for the keyboard: slash menu, drag handles, selection toolbar.

- **Editor** — TipTap block editor: headings, lists, to-do lists, quote, code, divider, **callout**, **toggle**, **columns**, **table**, **table of contents**, **inline + block equations (KaTeX)**, **breadcrumb**, **bookmark & embed** (YouTube/Vimeo/Loom/Spotify/Figma…), **synced view** (page transclusion), **@-mentions** (pages/people/dates), **file & image uploads** (paste, drag-drop or picker), rich inline formatting via a **selection bubble menu**, a keyboard-driven `/` slash menu, and **pointer drag-and-drop** reordering
- **Diagrams** — **Mermaid, PlantUML / UML and a visual UML canvas**: flowchart, sequence, class, state, ER, gantt, use-case, component and activity templates; source/preview switch; read-only rendering; drag/drop class/actor/use-case nodes; relationship types; per-node collaborator presence and live Yjs source/canvas editing. See docs/MERMAID.md
- **Editor blocks** — **linked database** (embedded read-only view of another database)
- **Block menu** — **Right-click any block**: Turn into, Colour, **Ask AI**, **Move up/down**, Duplicate, Comment, Copy link, **Edit equation**, Delete — plus a Notion-style caret and selection colour
- **Undo / redo** — Floating undo/redo controls in the editor
- **Backlinks** — **Linked references** — pages that @-mention the current page
- **Link previews** — `GET /api/link-preview` fetches OpenGraph metadata with an SSRF guard (http(s) only, DNS-resolved hosts and every redirect hop must be public), size cap and timeout
- **Page chrome** — Emoji icons, cover images (presets), breadcrumbs, page options menu (copy link, cover, export, move to workspace, lock, trash), favorites, **follow**
- **Page settings** — Full-width, small text and font (default/serif/mono), saved per page

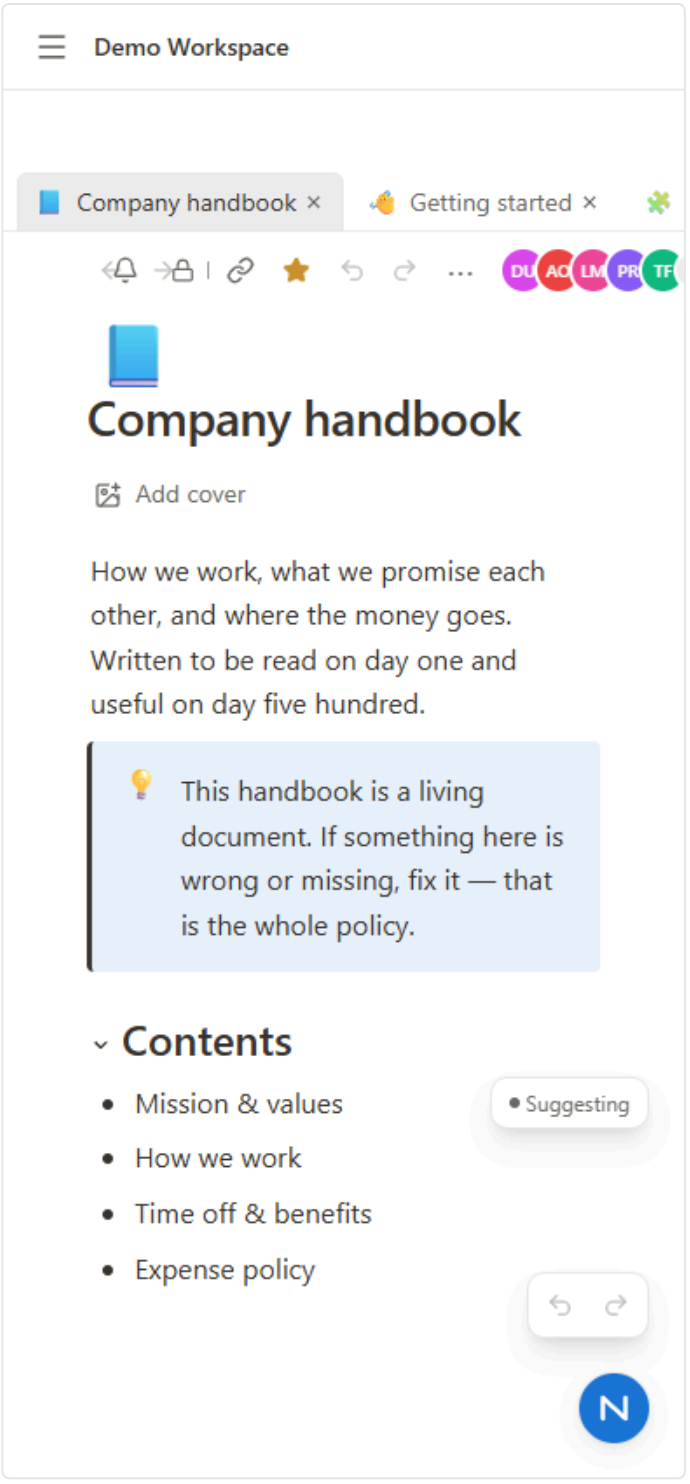


The block editor — headings, callouts, toggles, code, tables, backlinks

`/w/demo-workspace/cmvbj26430035ukloaxa7n387`



The slash menu
/w/demo-workspace/cmvbj26430035ukloaxa7n387



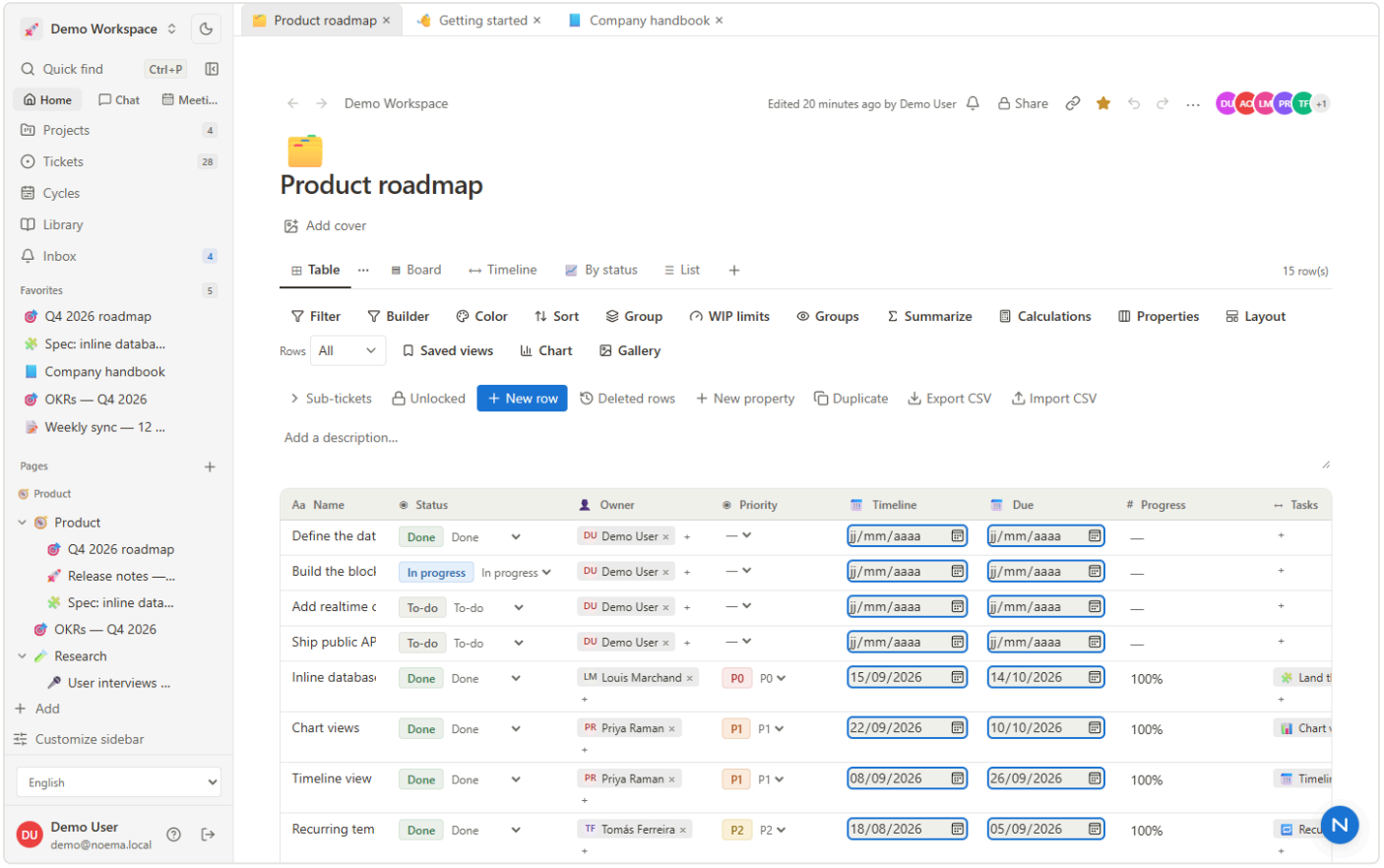
The same page on a phone viewport
/w/demo-workspace/cmvbj26430035ukloaxa7n387

CHAPTER 03

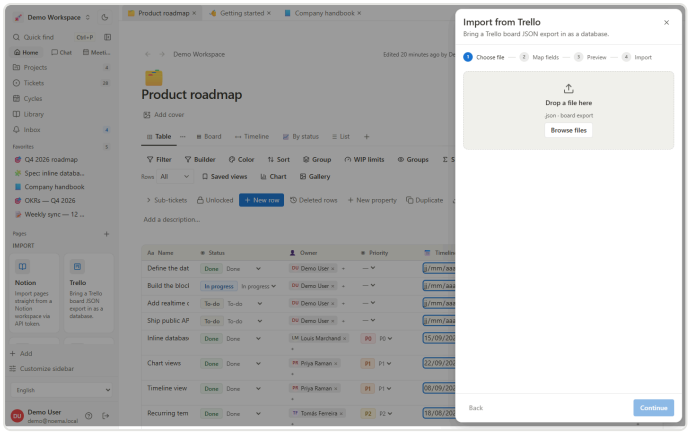
Databases and views

Typed properties over rows that are pages — one dataset, eight ways to look at it.

- **Databases** — Full-page databases with typed properties, **Table / Board / List / Gallery / Calendar / Timeline / Chart** views, **AND/OR + nested filters, sorts, grouping, conditional colors** (saved per view), view tabs with an “add view” menu, per-property menus (rename/duplicate/delete/formula), inline cell editing, templates, CSV import/export, open-row-as-page
- **Properties** — TEXT, NUMBER, SELECT, MULTI_SELECT, STATUS, DATE, CHECKBOX, URL/EMAIL/PHONE, PEOPLE, FILES, **FORMULA** (expression engine), **RELATION** (link to another database), **ROLLUP** (14 aggregates over related rows), and computed system fields (**CREATED_TIME/BY, LAST_EDITED_TIME/BY, UNIQUE_ID**)
- **Formulas** — FORMULA properties with an expression engine (`prop("Name")` , `+ - * / %` , comparisons, `if` , `concat` , `round` , `min/max` , `abs` , `length` , `upper/lower` , `contains` , `empty` , `now/today`)

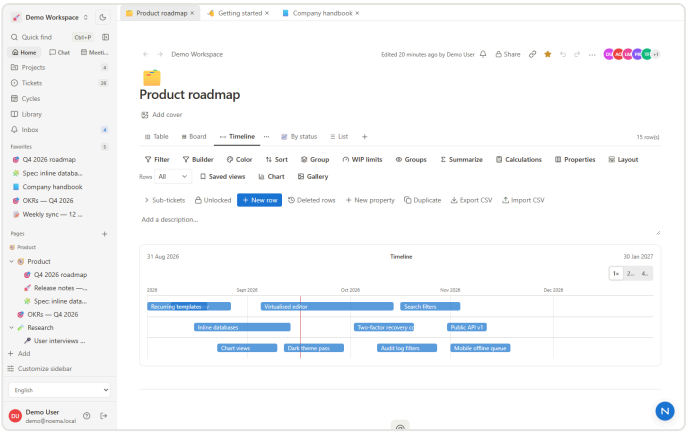


Database — table view with relations, a rollup and a percent property
`/w/demo-workspace/cmvbj264y003tuklo3pnyyl7g`



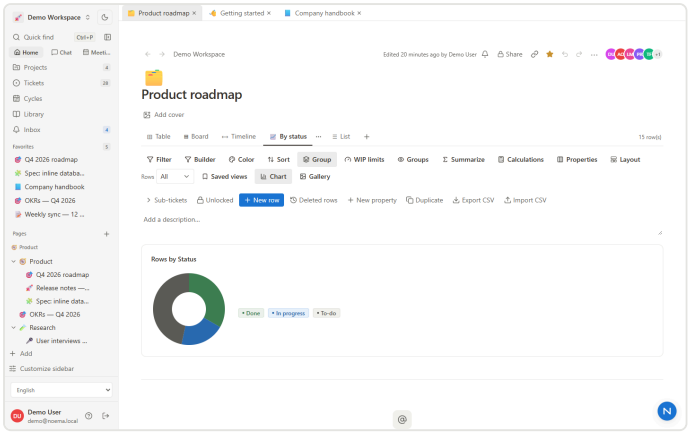
Database — board view grouped by status

`/w/demo-workspace/cmvbj264y003tuklo3pnyyl7g`



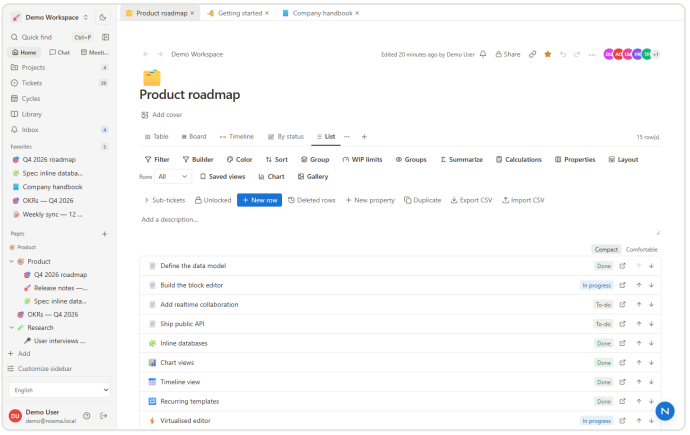
Database — timeline view

`/w/demo-workspace/cmvbj264y003tuklo3pnyyl7g`



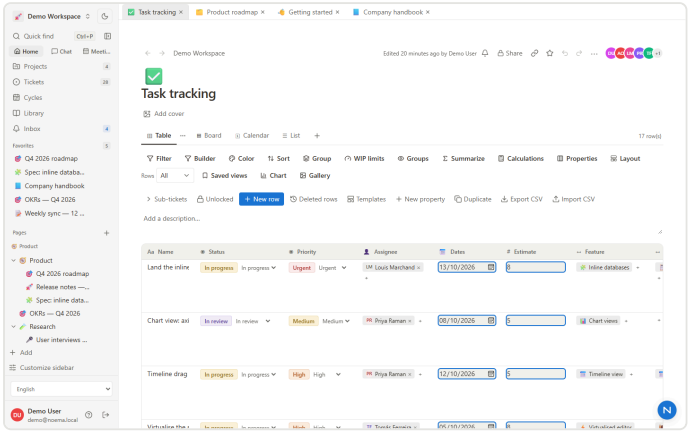
Database — chart view

`/w/demo-workspace/cmvbj264y003tuklo3pnyyl7g`



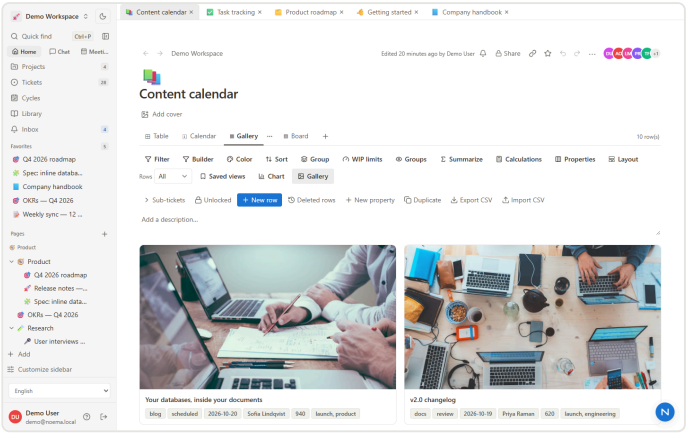
Database — list view

`/w/demo-workspace/cmvbj264y003tuklo3pnyyl7g`



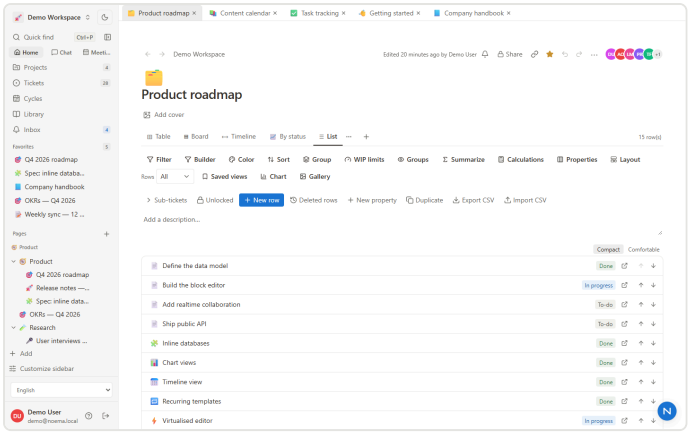
Database — calendar view

`/w/demo-workspace/cmvbj2660004puklofwqz7khi`



Database — gallery view with cover images

`/w/demo-workspace/cmvbj268k0071uklo86cyuxys`



A row, opened in the side peek

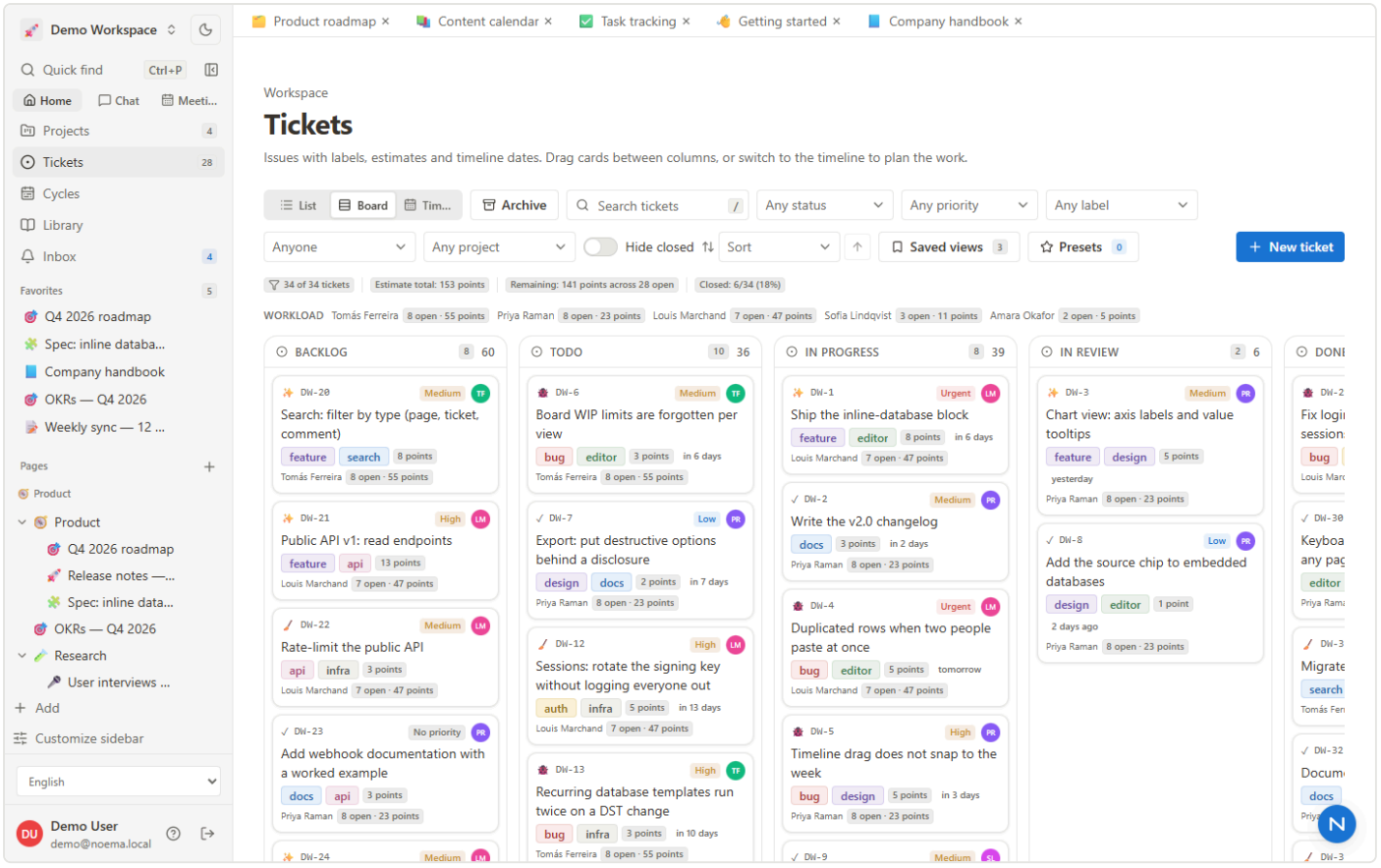
`/w/demo-workspace/cmvbj264y003tuklo3pnyyl7g`

CHAPTER 04

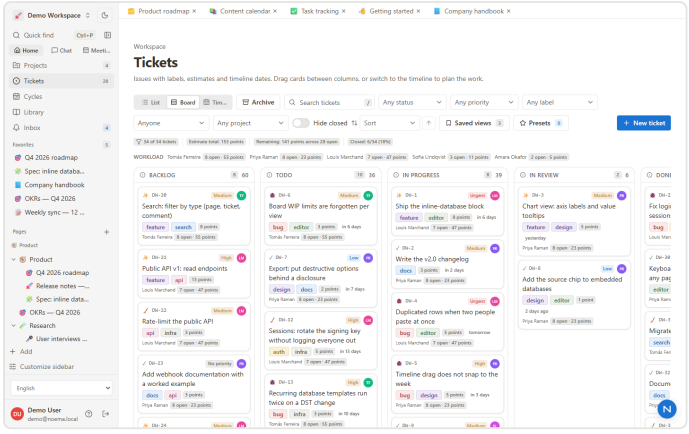
Planning — tickets, cycles and projects

A Linear-style issue tracker, sprints with a burndown, and projects with milestones — all on top of the same workspace.

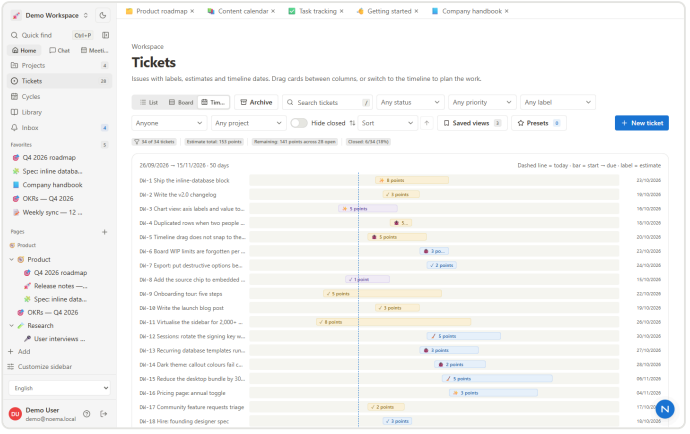
- **Keys** — Every ticket gets an ordered, workspace-prefixed key (`DW-1` , `DW-2` …) from an atomic counter
- **Labels** — **Workspace-level** label catalogue (name, colour, description, usage count) managed in Settings; any ticket can carry many, and you can filter by them
- **Fields** — type (task/bug/feature/epic/chore), status (Backlog → Todo → In progress → In review → Done/Cancelled), priority, assignee, reporter, project, milestone, parent/children
- **Estimates** — Points / hours / days per ticket, with **rollups**: total, remaining (open only), per board column and per project
- **Dates → Gantt** — Start & due dates drive the **Timeline** view: dated bars, a *today* marker, and estimate labels on each bar (undated tickets are shown flagged)
- **Views** — **List**, **Board** (drag between columns), **Timeline**, with search + status/priority/label/assignee/project filters and "hide closed"
- **Saved views** — Save the current filters, search and view type as a named workspace view; apply or delete it from the **Views** menu in the toolbar
- **Sub-tickets** — Break a ticket down from its drawer: parent link, child list with per-child estimates, a child estimate roll-up, inline *Add a sub-ticket*, and a sub-count badge in the list
- **Power features** — Relations (*blocks / blocked by / relates / duplicates*), comments, an **activity log** (created, status, labels, estimate, bulk, archived…), bulk edits, archive, and **Create ticket from page** in the sidebar's right-click menu
- **Integration** — Project pages show ticket counts/estimates; the chat assistant answers about tickets (`Reviewed tickets` , remaining estimate); `GET/POST /api/tickets`
- **Create a cycle** — Name, start/end dates and an optional goal (in the same dialog pattern as every other create flow; validated on the server)
- **Schedule work** — The ticket drawer has a **Scheduled in** selector; tickets keep their project, labels and assignee
- **Commitment** — Each cycle shows completed vs committed estimate, remaining points and a percentage, with a progress bar
- **Burndown** — Daily remaining points against the ideal straight line, drawn inline (SVG), out of the maths in `src/lib/cycles.ts`
- **Velocity** — The average completed estimate over the last six finished cycles, shown in the header
- **Delete** — Deleting a cycle unschedules its tickets — it never deletes them
- **Projects & milestones** — `/w/<slug>/projects` : projects with status/start/due dates, **milestones** (planned → in progress → done), a **Gantt timeline** with a today marker, notes attached per project (*New note in project*), and **client invitations** — a view-only guest scoped to that one project.



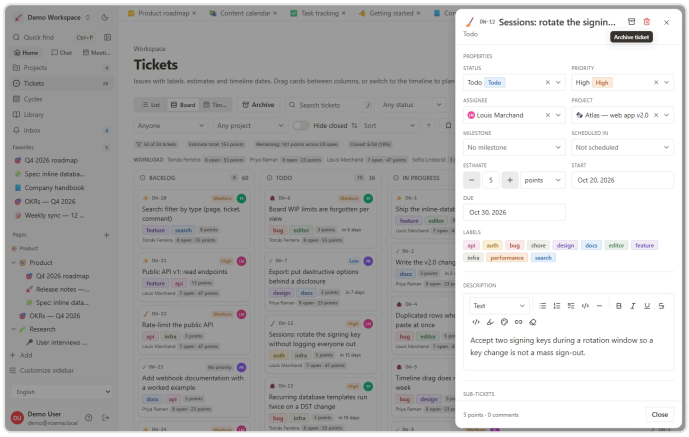
Tickets — list view with filters and saved views
`/w/demo-workspace/tickets`



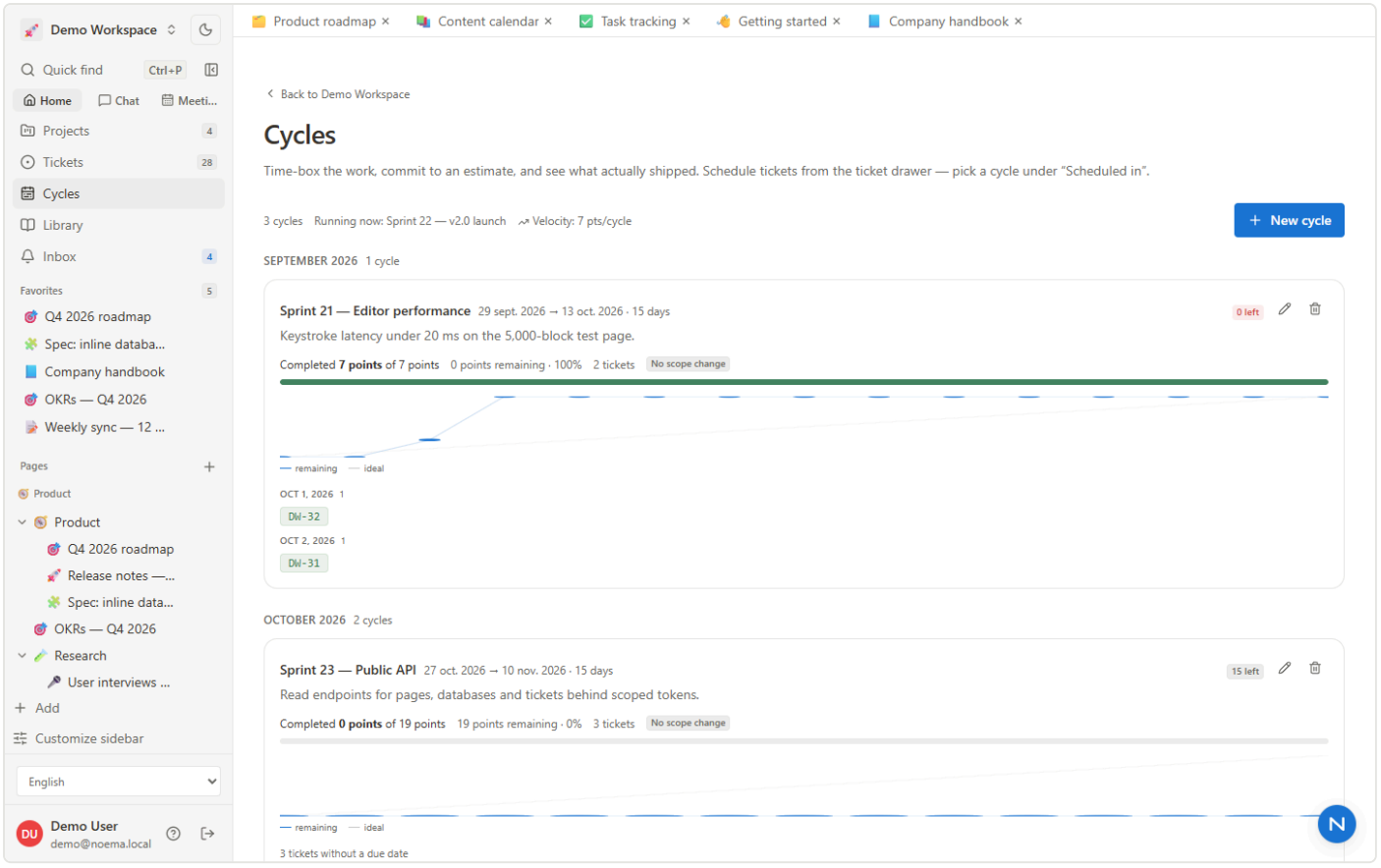
Tickets — board view
`/w/demo-workspace/tickets`



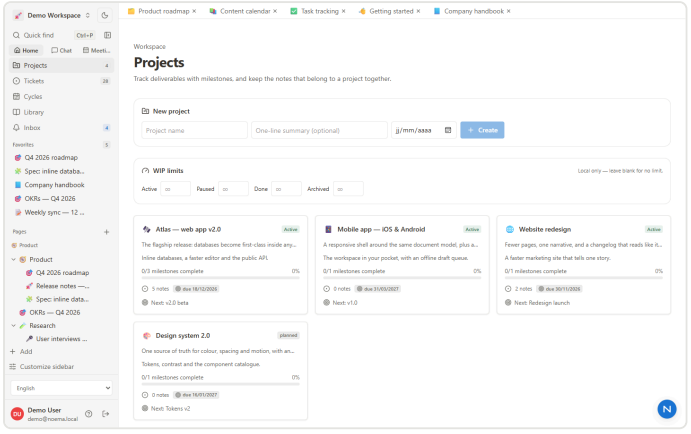
Tickets — timeline view
`/w/demo-workspace/tickets`



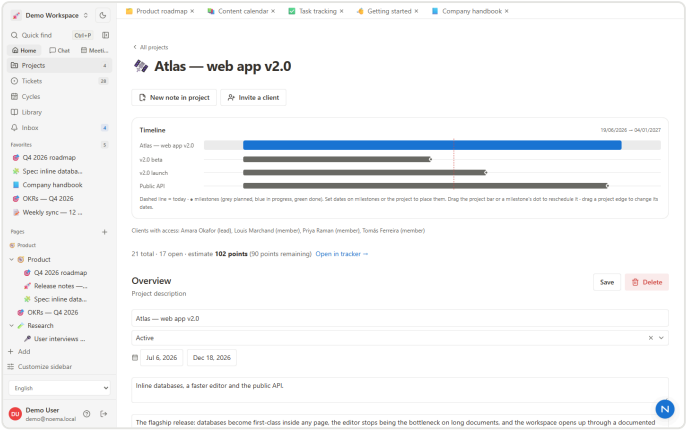
A ticket — description, activity, comments and linked work
`/w/demo-workspace/tickets`



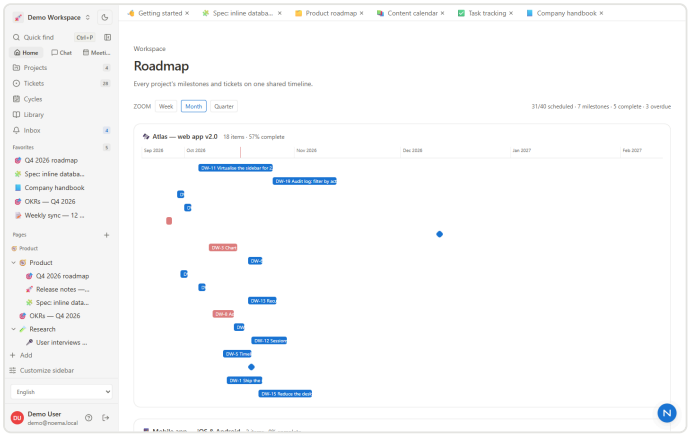
Cycles — burndown, scope and progress
`/w/demo-workspace/cycles`



Projects — progress, owners and target dates
`/w/demo-workspace/projects`



A project — milestones and linked tickets
`/w/demo-workspace/projects/cmvbj261z001duklosro2newh`



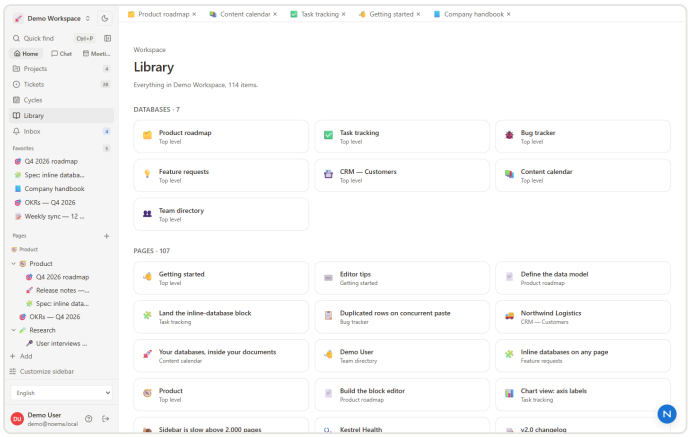
Roadmap — the workspace-level timeline
`/w/demo-workspace/roadmap`

CHAPTER 05

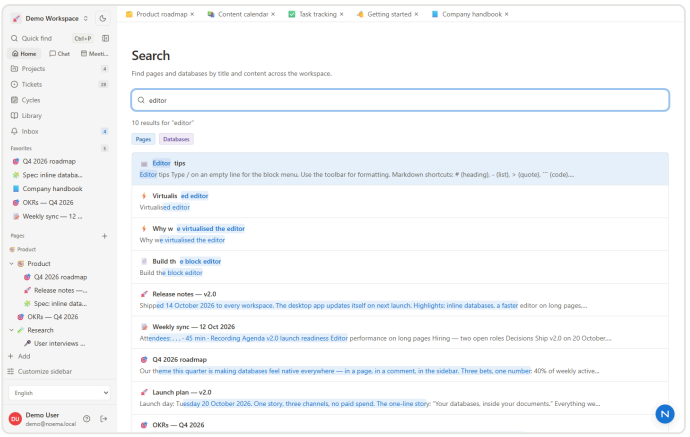
Finding and organising

Postgres full-text search, a library of everything, an inbox, templates — and a trash that restores.

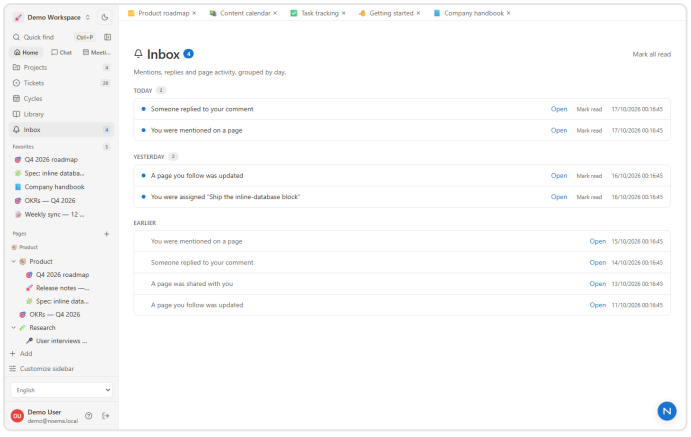
- **Search** — Full-text search across page titles and body content
- **Library** — `/w/<slug>/library` — every database and page in one browsable grid
- **Inbox** — Notification inbox with unread badge and “mark all read”
- **Home & templates** — A greeting home with **Empty page / Empty database** cards, a **workflow template gallery** (Task tracking, Projects, Skills, Document center, Brainstorm, Meeting notes) with mini previews, and “Other templates”; `POST /api/templates`
- **Templates** — **Template button** block + database templates with **recurrence** (daily/weekly/monthly) materialized by `POST /api/cron/templates`
- **Database templates** — Save any row as a template; create rows from templates



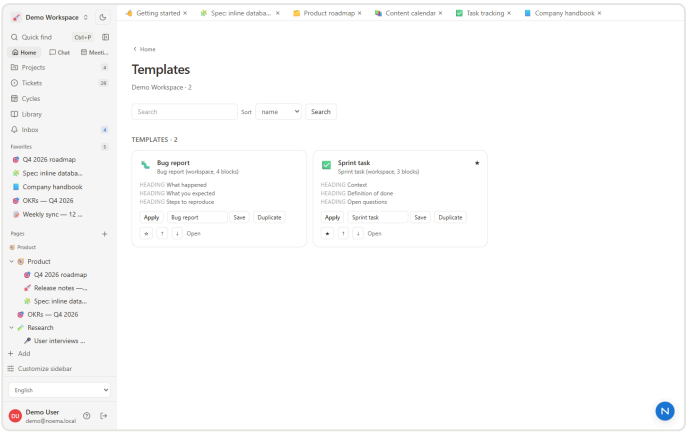
Library — every database in the workspace
`/w/demo-workspace/library`



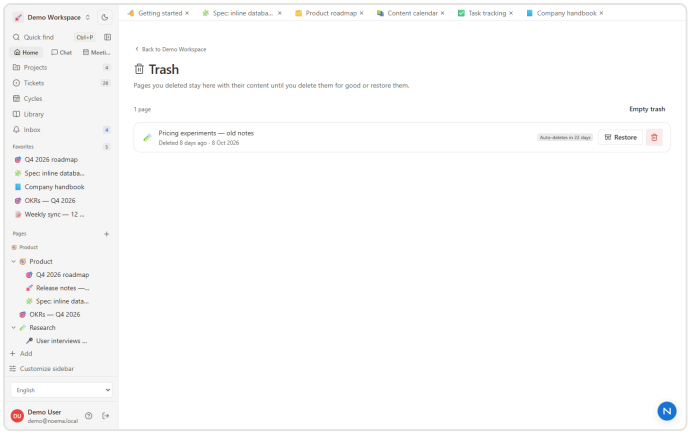
Search — full text, scoped to what you can see
`/w/demo-workspace/search?q=editor`



Inbox — mentions, comments and updates
`/w/demo-workspace/inbox`



Templates — saved templates and the built-in library
`/w/demo-workspace/templates`



Trash — restore, or delete for good

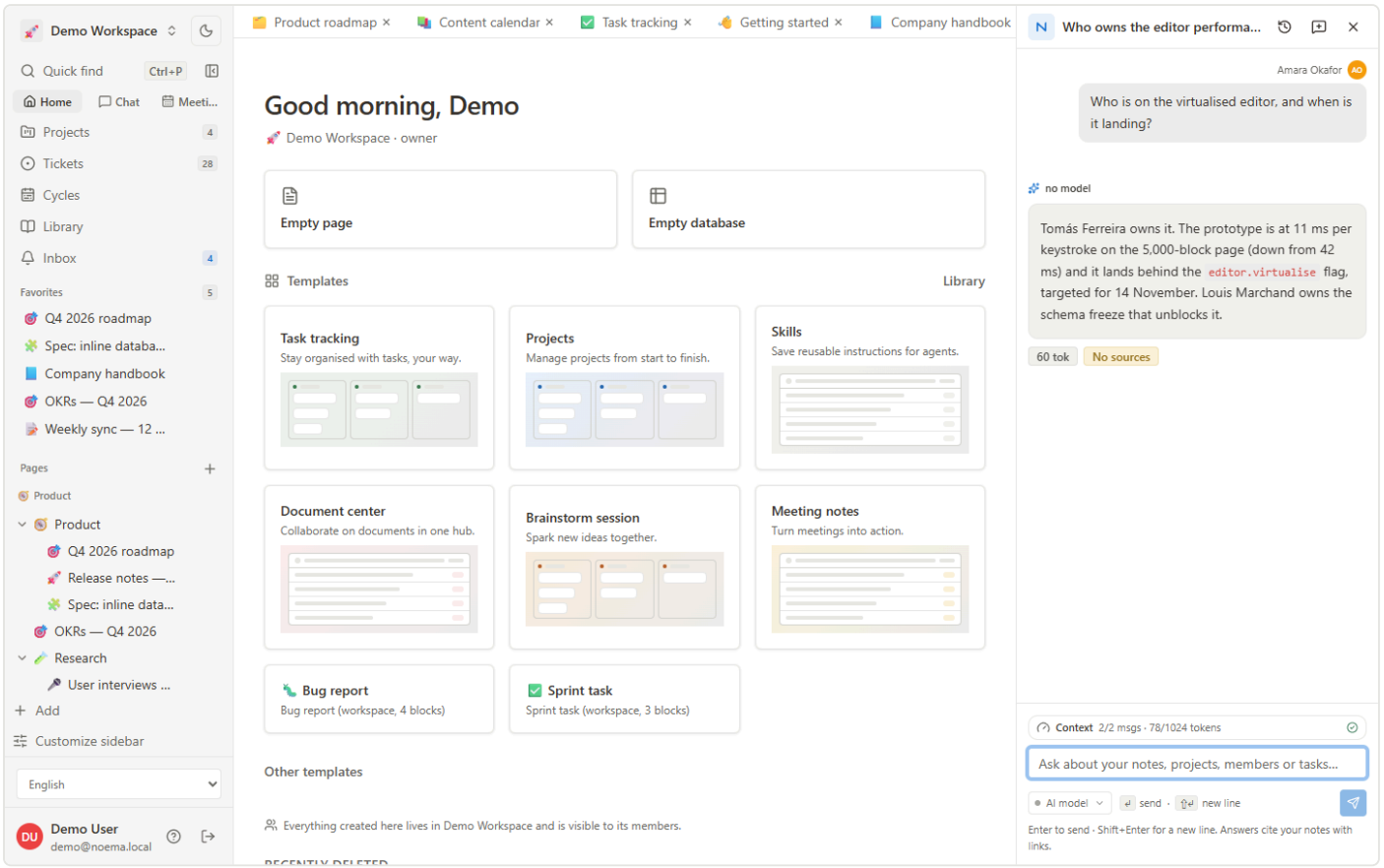
`/w/demo-workspace/trash`

CHAPTER 06

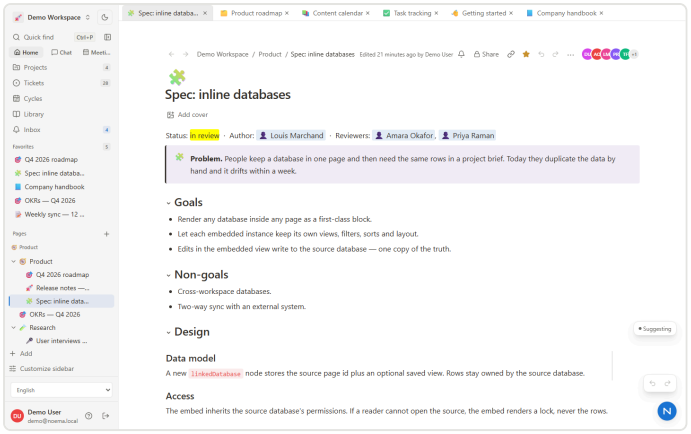
Collaboration

Comments anchored to text, workspace chat grounded in your own notes, meetings, live cursors and restorable history.

- **Comments** — Page comments, threaded replies, resolve, delete, notifications on reply
- **Discussions** — **Workspace chat**: draft-first threads (no empty rows — the thread is created on the first message), first message names it, notes can be pinned to a discussion, and `/ask …` gets a grounded AI answer; `GET/POST /api/chat`
- **Meetings** — **Calendar subscriptions** — paste an ICS URL, events are imported/refreshed, and one click turns an event into a **meeting-notes page**; `GET/POST /api/calendar`
- **Realtime** — **Optional Yjs collaboration** (`npm run collab` + `NEXT_PUBLIC_COLLAB_URL`) with awareness/live cursors and a “Live · N” indicator
- **Version history** — **Slide-over panel** (no longer a long list in the page body): who changed what, relative time, character delta, “Renamed to …”, and one-click **Restore**

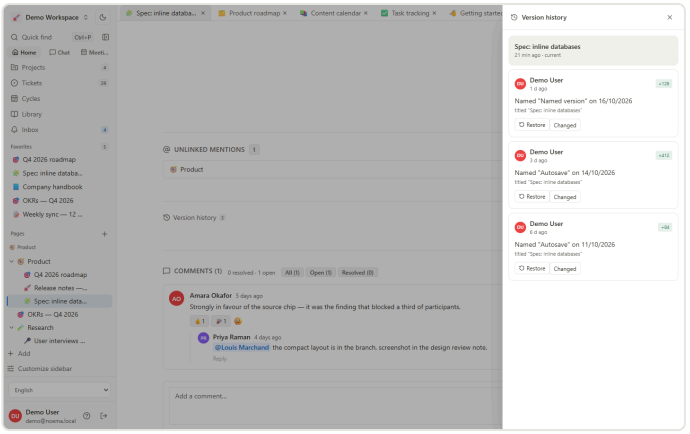


Chat — a workspace conversation, with the notes it cites
`/w/demo-workspace?chat=cmvbj271301dquklo9foe9y0n`



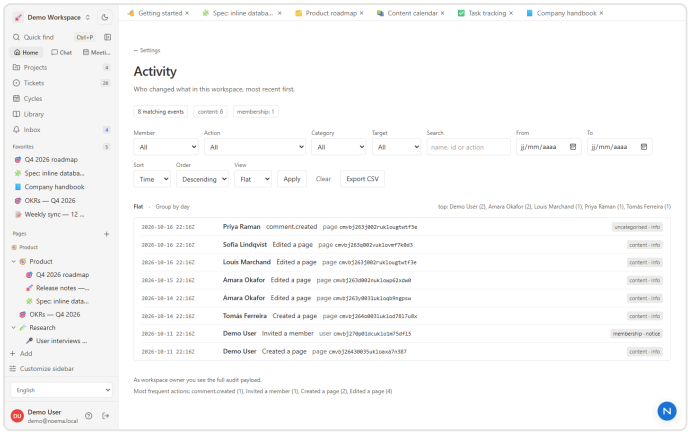
Comments and page discussion

/w/demo-workspace/cmvbj263j002ruklougtwtf3e



Version history — every save, restorable

/w/demo-workspace/cmvbj263j002ruklougtwtf3e



Audit log — who changed what

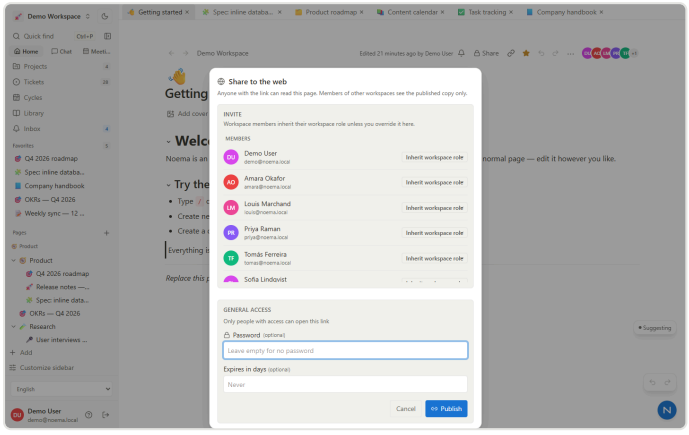
/w/demo-workspace/audit

CHAPTER 07

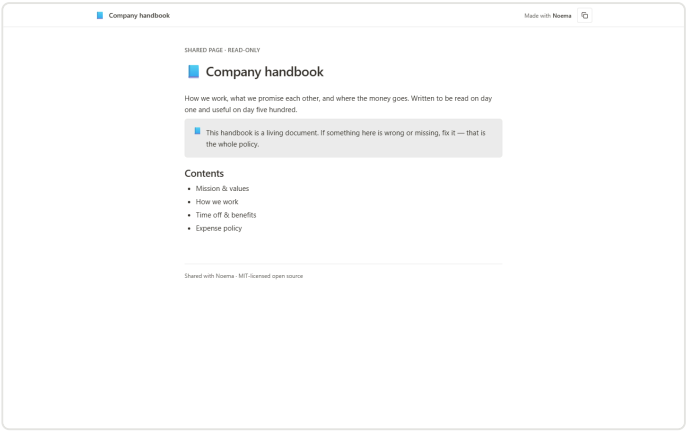
Sharing and permissions

Publish a page to the open web, and decide exactly who can open what inside the workspace.

- **Sharing** — **Share to web** — publish a page read-only at `/s/<token>` with optional **password + expiry**; unpublish; per-member share levels and **email invitations** (see Permissions)
- **Page permissions** — Per-member page access (**Can edit / Can view / No access**) that overrides the workspace role for that page
- **Roles** — Owner · admin · member · **guest** — enforced on the server, never only in the UI
- **Guest page-scoping** — A guest opens **only** the pages shared with them: an explicit `PagePermission`, a page in a teamspace they belong to, or a page they created — plus ancestors so the tree still renders. Applied to the sidebar tree, search and **the page route** (a direct URL to an unshared page is a 404, not a leak). Guests cannot edit, invite or upload
- **Teamspaces** — Members see and search only their teamspaces; invites are emailed
- **Page shares** — Per-member levels (inherit · view · comment · edit · full) and public links with an optional password and expiry



Share — public link, permissions and export
`/w/demo-workspace/cmvbj2631002hukloi0z5s75o`



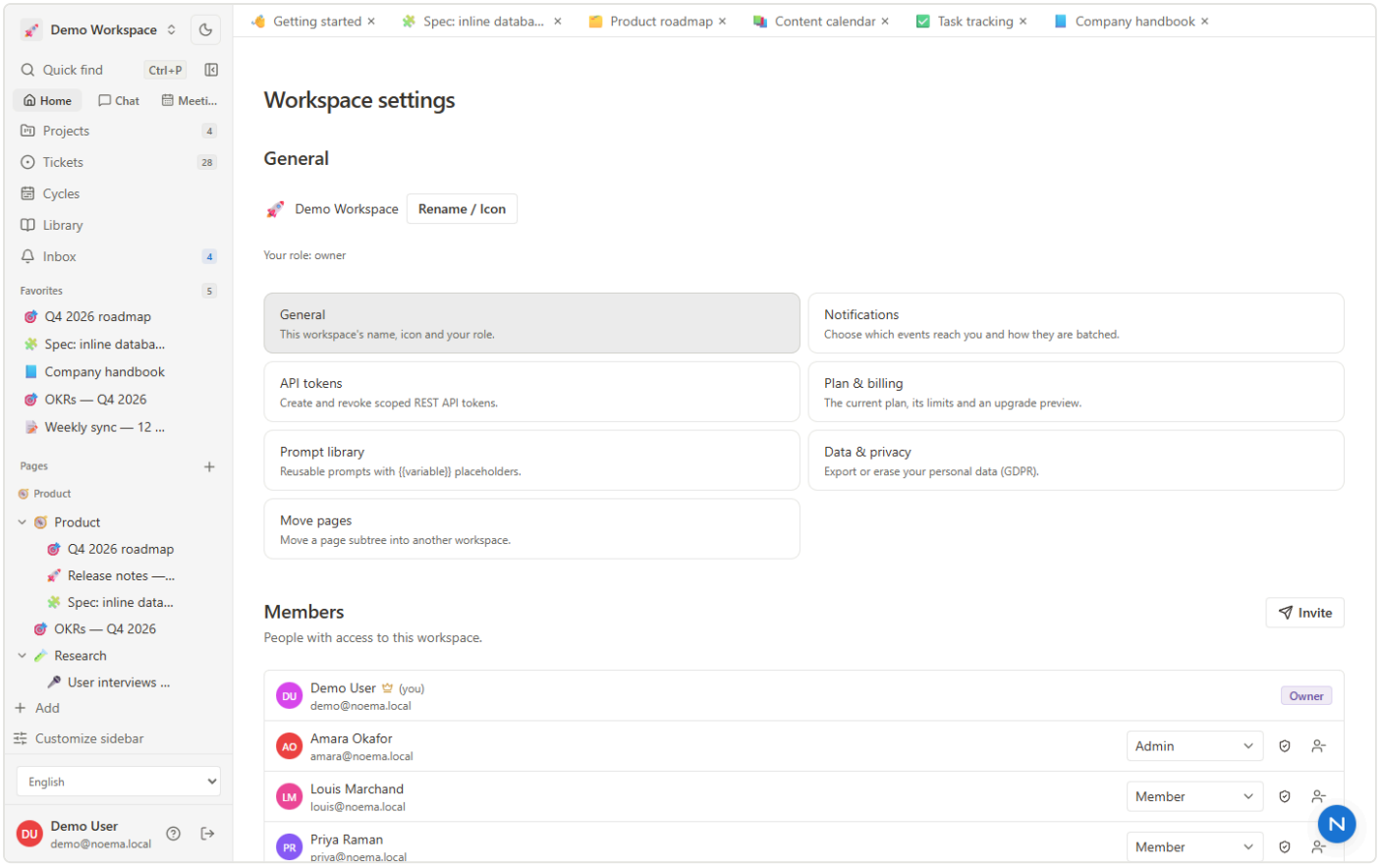
A published page, as a visitor sees it
`/s/demo-handbook-share`

CHAPTER 08

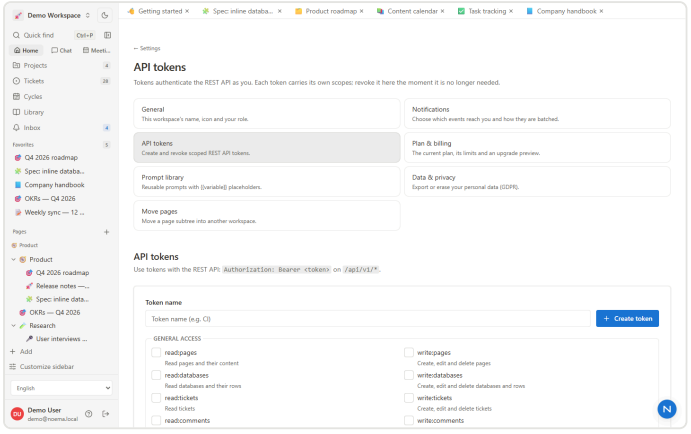
Platform, AI and administration

An HTTP API, OAuth apps, signed webhooks, bring-your-own-model AI, importers, and the settings that hold it together.

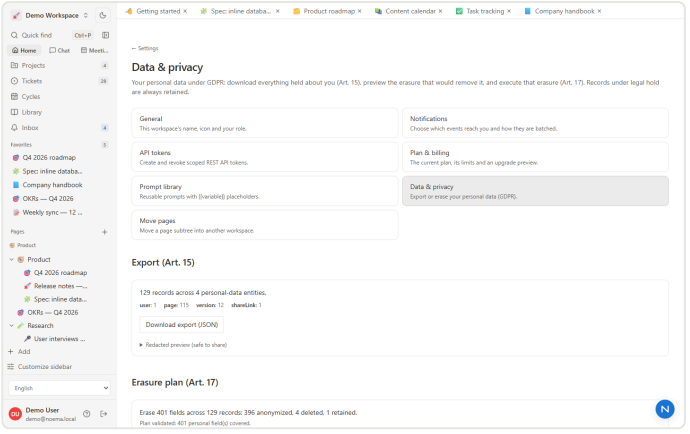
- **Public API** — `GET/POST /api/v1/pages`, `GET /api/v1/pages/:id`, `GET /api/v1/search`, `GET /api/v1/workspaces` — authenticated with **Bearer API tokens** (hashed at rest, managed in workspace settings); the token's **scope grant is enforced** (`read:pages` for the read routes, `write:pages` for creating pages)
- **MCP** — `/api/mcp` - 43 tools for pages, blocks, databases, comments, tickets, projects, cycles, templates, workspace admin and exports; token-scoped, read-first, write/admin opt-in (`allowWrite` / `allowAdmin`), resources for page/database reads. See docs/MCP.md
- **OAuth apps** — Register an app in settings; **authorization-code flow** (`/oauth/authorize` → `/oauth/token`) issues Bearer API tokens
- **Webhooks** — **HMAC-SHA256-signed** POST deliveries on `page.created`, `page.updated`, `row.created`, `comment.created` ; managed in workspace settings
- **Webhook log** — Delivery outcomes recorded (`WebhookDelivery`) with **retries** (3 attempts, backoff), status/attempts/error shown in settings
- **AI** — `POST /api/ai` (OpenAI-compatible: OpenAI, **DeepSeek**, Groq, Ollama…) + an **Ask AI** block (ask + summarize) + a database **AI autofill** property + **AI skills** (`POST /api/ai/skills`); degrades to 503 guidance when unconfigured
- **AI grounding** — Ask answers are grounded in your workspace: prompts are reduced to keywords and pages scored by relevance (`src/lib/retrieval.ts`)
- **AI skills** — Selection-toolbar **Ask AI**: Improve writing, Proofread, Reformat, Make shorter, Explain, Summarize, Translate
- **Automations** — **Button properties** (set another property to a value on click) and **page locking** (read-only pages/databases)
- **Security** — bcrypt passwords, HS256 JWT session cookies, **2FA (TOTP)** with any authenticator app, API tokens hashed at rest, **fixed-window rate limiting** on every API route
- **Import / Export** — Export a page as **Markdown / JSON / HTML**, a database as **CSV**, the whole workspace as a **.zip**; import **Markdown** or **CSV**
- **Exports** — plus **PDF** (via `pdfkit`)
- **Importers** — **Trello JSON** → database (lists → status options), **Markdown .zip** → nested pages, **Word .docx** → page, Markdown file, **Notion API** (`POST /api/import/notion` — integration token + nested child pages)
- **i18n / RTL** — 5 locales (en/fr/es/de/ar) with correct `lang` / `dir` , translated core navigation, and a language switcher
- **Accessibility** — Visible focus rings, a **skip-to-content** link, `lang` / `dir` from locale, labelled landmarks/navs/controls, and a **WCAG contrast-audited palette** (`npm run check:a11y`)
- **Workspace settings** — Rename, member list, API tokens, workspace export



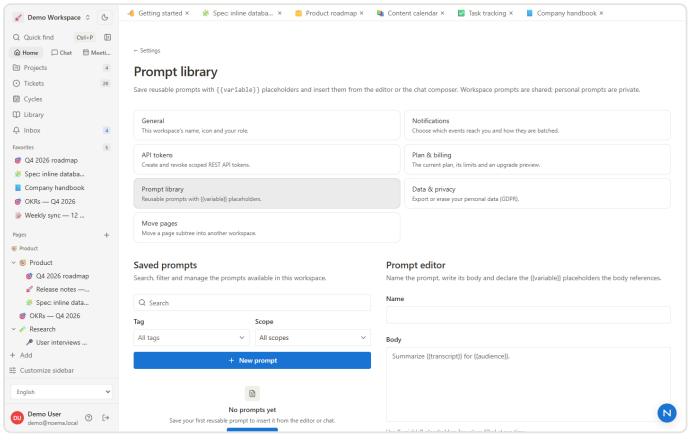
Settings — general, members, roles and invites
`/w/demo-workspace/settings`



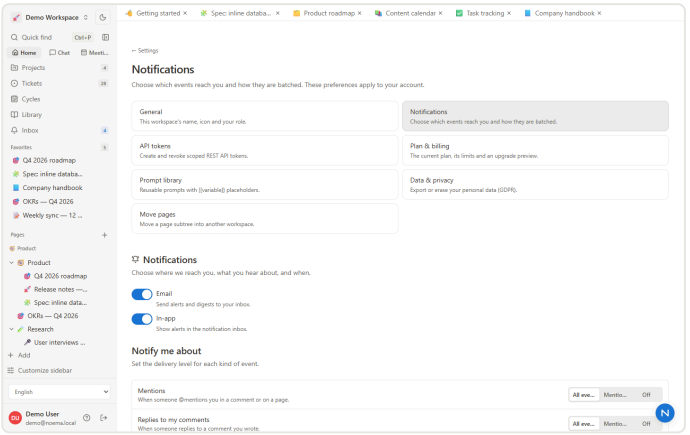
Settings — API tokens
`/w/demo-workspace/settings/tokens`



Settings — data, export and privacy
`/w/demo-workspace/settings/data`



Settings — the prompt library
`/w/demo-workspace/settings/prompts`



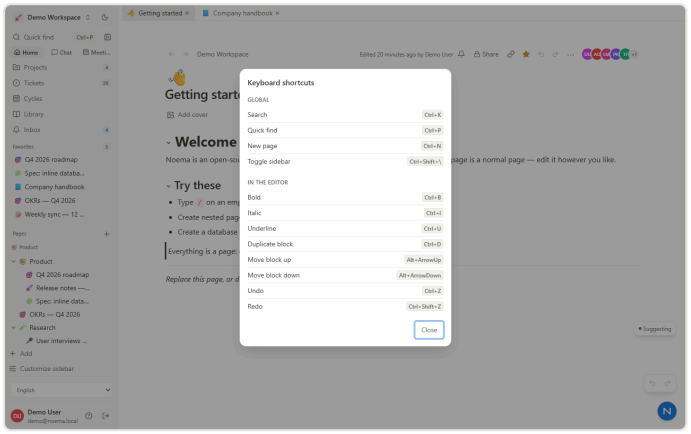
Settings — notification preferences
`/w/demo-workspace/settings/notifications`

CHAPTER 09

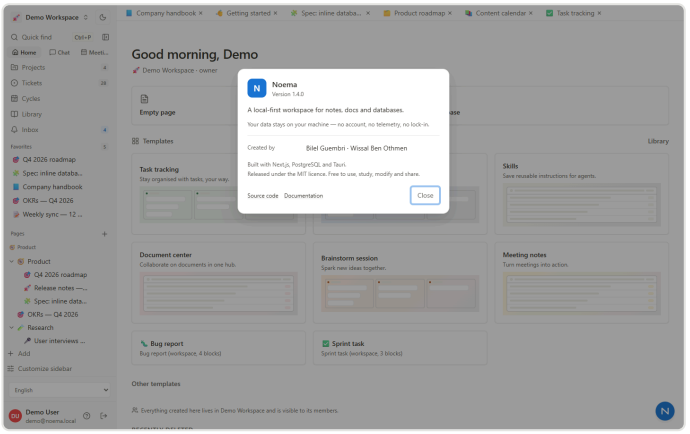
The desktop app

The same codebase as a per-user Windows installer: a Tauri 2 shell around a bundled Node runtime and an embedded PostgreSQL.

- **Install** — The NSIS installer (`Noema_1.4.1_x64-setup.exe`) installs per user into `%LOCALAPPDATA%\Noema` , and uninstalls from Add/Remove programs. WebView2 is bootstrapped by the installer itself
- **Your data** — `%APPDATA%\app.noema.desktop` — the database, uploads, the session secret and preferences. Uninstalling keeps it unless you tick *delete application data*
- **Build it** — `npm run desktop:bundle` → `src-tauri/target/release/bundle/{nsis,msi}`
- **How it boots** — The shell starts the bundled server on a free loopback port and opens the window on it. The port is a preference, not a contract: it survives a port change because nothing outside the shell depends on the number
- **One codebase** — The desktop build is the site build with `NOEMA_STANDALONE=1` — the embedded database replaces Postgres and the shell owns the lifecycle. Nothing is forked
- **Native menu** — A real application menu — File, Edit, View, Help — wired to the same commands the in-app palette runs, including Help → **About** and Help → **Keyboard shortcuts**.
- **Diagnostics** — Help → **Diagnostics** shows the shell, server, database and storage locations, and can take a backup without leaving the window.
- **Nothing to install** — No Docker, no Postgres, no account: the embedded engine starts with the app and keeps your data in `%APPDATA%\app.noema.desktop` .



Keyboard shortcuts — the same dialog the desktop menu opens
`/w/demo-workspace/cmvbj2631002hukloi0z5s75o`



About — version, credits and licence, from Help → About
`/w/demo-workspace`

CHAPTER 10

Quality and engineering

The parts a screenshot cannot show: the gates that keep the tree honest.

- **Quality** — **Vitest** unit tests for the pure logic (block serialisation, database query engine), **Playwright** e2e, `tsc --noEmit`, ESLint, `npm run check:fast` inner-loop gate and `npm run check:coherence` drift gate
- **Drift checks** — `npm run check:coherence` (docs vs code), `check:i18n` (locale parity), `check:secrets`, `check:a11y` (contrast), `check:design` (design-system rules) and `check:embedded-db` (the packaged database, end to end).
- **Honesty** — Every feature area is tracked row by row in `docs/FEATURE_MATRIX.md`, and `npm run docs:gaps` generates `docs/MISSING_FEATURES.md` from it — so the two cannot disagree.

Running it yourself

The web app runs on Postgres; the desktop app needs nothing else.

```
docker compose up -d             # Postgres 16 on localhost:5433
npm install
npx prisma migrate reset --force # schema, demo data, search index
npm run dev                      # http://localhost:3000

npm run desktop:bundle           # → Noema_1.4.1_x64-setup.exe
```

Sign in with `demo@noema.local` / `password123` — the workspace in these screenshots.

Where to read next

The README for the summary, `docs/FEATURE_MATRIX.md` for the row-by-row tracker, `docs/MISSING_FEATURES.md` for what is unfinished, `docs/DESKTOP.md` for packaging and the embedded database, and `docs/specs/` for the long-form specifications.

How this document is made

`npm run features:pdf` renders this file from the README's own feature tables and the screenshot report. Change a feature and the next build of the sheet changes with it.
